LAB
13

Operating Systems · FCIM / FAF · UTM · 2026-2027

Semaphores in Your Own Kernel

B **Track B: build your own small OS (xv6) · 90 minutes**

AT A GLANCE

Part 1	The numbers and the user side	~20 min
Part 2	The kernel side	~40 min
Part 3	A semaphore as a lock: mix	~20 min

Goal

In Labs 11 and 12 you USED semaphores from the C library. Today you BUILD them, inside the xv6 kernel, as three new system calls: sem_init, sem_down and sem_up (numbers 24, 25, 26). You will see that a semaphore is only a number, a spinlock that protects it, and the kernel's sleep and wakeup. Processes that wait really go to sleep: they use no CPU.

At the end this works:

TERMINAL

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
ping 3
  pong 3
semtest: OK
```

Before you start

- Use your ~/xv6-riscv tree, with or without trace from Lab 10. Check that it builds and boots: make qemu, then Ctrl-A X.
- Every block of code is complete. After each part there is a check: do not go on until it passes.

Careful: numbers and the sleep function in our xv6

The new calls get 24, 25 and 26 (22 is sync, 23 is kept for trace from Lab 10). With or without trace, the same numbers work: an empty place 23 in the table does no harm. Our kernel sleeps in two steps, sleep_prepare(chan) then sleep(), not with sleep(chan, lock) as in the xv6 book: copy the code below, not the book.

Minimum to pass

Parts 1 and 2: semtest prints the child after the parent and ping/pong in turn, then semtest: OK.

PART 1 The numbers and the user side ~20 min

The same recipe as trace in Lab 10, three times. Semaphores are numbered 0 to 7 and shared by all processes.

1.1 kernel/syscall.h: the numbers

FILE kernel/syscall.h

```
#define SYS_sync   22
#define SYS_sem_init 24      // new
#define SYS_sem_down 25      // new
#define SYS_sem_up   26      // new
```

1.2 user/user.h: the declarations

FILE user/user.h

```
int sync(void);
int sem_init(int, int);  // new: id, start value
int sem_down(int);       // new: take one, or sleep
int sem_up(int);         // new: give one back
```

1.3 user/usys.pl: the stubs

FILE user/usys.pl

```
entry("sync");
entry("sem_init");       # new
entry("sem_down");       # new
entry("sem_up");         # new
```

1.4 user/semtest.c: the test

Test 1: the child must wait for the parent (semaphore 0 starts at 0). Test 2: ping and pong take turns, with two semaphores that pass the turn to each other.

FILE user/semtest.c

```
// semtest.c: two tests for the kernel semaphores (Lab 12).
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
```

```
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("  pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

1.5 user/mix.c: two processes on one screen

You need it in Part 3. Create it now so the Makefile has everything:

FILE user/mix.c

```
// mix.c: two processes print five lines each, one letter per write().
//   mix        no lock: the letters of the two processes can mix
//   mix lock   semaphore 3 used as a lock: every line stays whole
#include "kernel/types.h"
#include "user/user.h"

void
lines(char *s, int uselock)
{
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int
main(int argc, char *argv[])
{
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

1.6 Makefile: put both programs on the disk

In UPROGS, after $U/_zombie\ (lines start with a TAB):

FILE Makefile

```
	$U/_zombie\
	$U/_semtest\
	$U/_mix\
```

Check 1: the calls reach the kernel, which does not know them yet

make qemu, then semtest. The first lines must be exactly these:

TERMINAL

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
```

After them you see ping and pong mixed with the error lines. Look at the line child: it comes BEFORE the line parent:. Without semaphores the child ran first. That is exactly what Part 2 fixes.

PART 2 The kernel side ~40 min

2.1 kernel/sem.c: the semaphores (a new file)

Read it top to bottom before you type it. sem_down is the heart: while the value is 0, the process registers the address of the semaphore as the thing it waits for, lets go of the spinlock, and sleeps. sem_up adds 1 and wakes up everyone who sleeps on that address.

FILE kernel/sem.c

```
// sem.c: counting semaphores inside the xv6 kernel (Lab 12).
// A small table of NSEM semaphores, shared by every process.
// Each one is a number plus a spinlock that protects the number.

#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;   // a zeroed spinlock is a free spinlock
  int value;              // how many may still pass
};

static struct sem sems[NSEM];

// sem_init(id, value): give semaphore id its start value.
uint64
sys_sem_init(void)
{
```

```
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

// sem_down(id): take one unit; if there is none, sleep until there is.
uint64
sys_sem_down(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);       // I will sleep on the address of s
    release(&s->lock);      // never sleep holding a spinlock
    sleep();                // returns at once if sem_up came in between
    acquire(&s->lock);
    if (killed(myproc())) { // kill must still work on a waiting process
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): give one unit back and wake whoever sleeps on s.
uint64
sys_sem_up(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  s->value++;
  wakeup(s);
  release(&s->lock);
  return 0;
}
```

2.2 Makefile: compile sem.c into the kernel

In the OBJS list at the top, after $K/sysproc.o \ (lines start with two spaces):

FILE Makefile

```
  $K/sysproc.o \
  $K/sem.o \
```

2.3 kernel/syscall.c: the table

FILE kernel/syscall.c

```
extern uint64 sys_sync(void);
extern uint64 sys_sem_init(void);   // new
extern uint64 sys_sem_down(void);   // new
extern uint64 sys_sem_up(void);     // new
...
  [SYS_sync]    = sys_sync,
  [SYS_sem_init] = sys_sem_init,     // new
  [SYS_sem_down] = sys_sem_down,     // new
  [SYS_sem_up]   = sys_sem_up,       // new
```

Check 2: semtest

Ctrl-A X, make qemu, then:

TERMINAL

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
ping 3
  pong 3
semtest: OK
```

The parent waits 10 ticks before it lets the child go, yet the child prints second: it was asleep in sem_down. Ping and pong alternate strictly, three times each.

If it does not work

Still "unknown sys call 24": the table entries of 2.3 are missing. Build error "undefined reference to sys_sem_init": sem.o is not in OBJS (2.2). Build error about sleep: you wrote sleep(s, &s->lock) from the book; use sleep_prepare(s), release, sleep(), acquire, as in 2.1. semtest hangs: sem_up does not call wakeup(s), or sem_down sleeps on another address than the one sem_up wakes.

PART 3 A semaphore as a lock: mix ~20 min

mix starts two processes that each print five lines, one letter per write() system call. Run it twice without a lock, then with one (semaphore 3, started at 1). make qemu uses 3 CPUs, so the two processes really run at the same time.

Real output without the lock (yours will be mixed differently):

TERMINAL

```
$ mix
AAAAAAAbAbbAbAbbbbbbAbAbAbbAbAbbbbbbbbbbbbbbbbb
AbAbbbbbbbAbbAbAbAbAbAbbbbbbbbbbbbbbbbbAb
AAAAbbAA
bbbbbbbbbbbAbAAAAbbbAbAbAbAbAbAbAbAbAbAbAbAbA
AbAbAbAbAbAbAbAbAbbbAbbAb
```

With the lock:

TERMINAL

```
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Observe (write in notes.txt)

1. Without the lock, why do the letters mix? (How many system calls does one line cost?)
2. With the lock every line is whole. Which two lines of mix.c make one line a critical section?
3. In sem_down the test is "while (s->value == 0)", not "if". Why can a process wake up and still find 0?
4. sem_down calls release(&s->lock) BEFORE sleep(). What would go wrong if it slept holding the spinlock?

Optional (not graded)

Add a fourth call, sem_trydown(id) (number 27): like sem_down, but if the value is 0 it returns -1 at once instead of sleeping. Write a tiny user program that shows both answers.

What you hand in

- Your modified xv6 tree (committed to git), with semtest printing semtest: OK.
- notes.txt with the four Observe answers.

Criterion	Points
Part 1: Check 1 reached (unknown sys call 24)	**3 p**
Part 2: semtest correct (order and ping/pong)	**4 p**
Part 3: mix with and without lock, Observe answered	**3 p**
Total	**10**